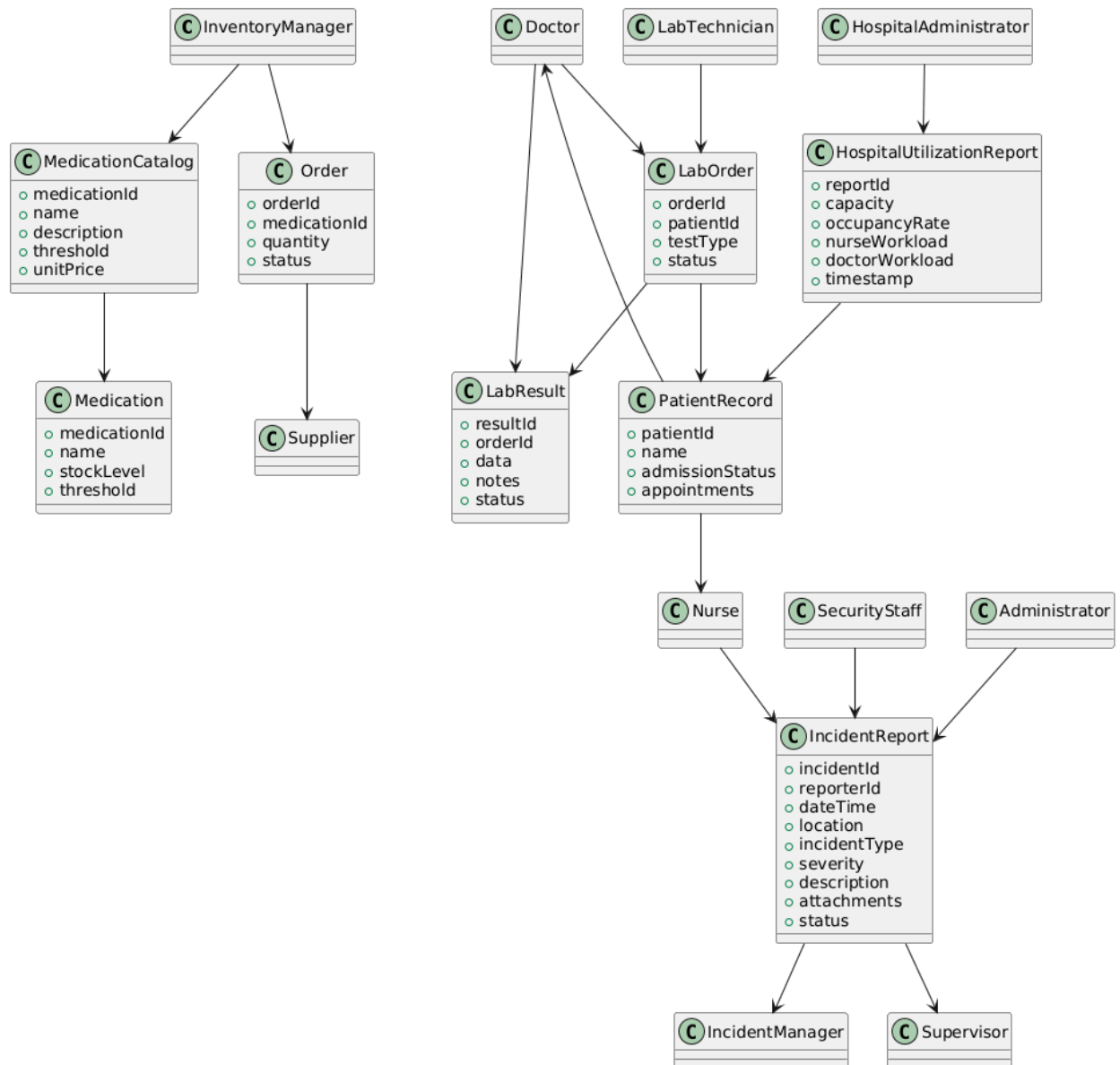


Hospital ERP System

Tirmidi M., Harley P., Josh S., Cayden O., Aidan S.

Iteration Three

Domain Model



Justification

Only keeping track of one hospital's inventory, so there will not be a class to aggregate medicationStock. This choice reduces the complexity of our solution.

The design for the Generate Hospital Utilization Report use case follows core object-oriented principles, especially Information Expert and the Controller pattern. The HospitalSystem class acts as the controller, coordinating the workflow and delegating tasks to the subsystems that already own the necessary data. This keeps high-level logic centralized while preventing unnecessary coupling.

Hospital-level occupancy metrics are handled by the Hospital class because it maintains total bed count and admitted patient lists. Patient retrieval responsibilities belong to PatientFileManager, the expert in loading and accessing patient records. Each PatientRecord provides its own admission status and appointments, preserving encapsulation and avoiding duplicate logic.

Doctor workloads are computed by DoctorManager, which owns the doctor roster and appointment data, while nurse workloads are handled by EmployeeViewer, the expert on staff assignments. Final results are assembled into a UtilizationReport, improving cohesion by separating report content from orchestration logic. Overall, this structure ensures low coupling, high cohesion, and logically distributed responsibilities.

The HospitalSystem uses IncidentReportController to coordinate the workflow, while IncidentReportService handles creating and saving reports. PatientFileManager validates patient records, and reporter information is captured directly with the Employee model. Each IncidentReport stores the patient ID, reporter ID, description, and timestamp, keeping data consistent and encapsulated. This design ensures clear responsibilities, low coupling, and smooth integration with hospital workflows.

The OrderMedication uses both the information expert and controller design principles. OrderMedication uses the InventoryService as the information expert because it owns and manages inventory data, medication orders, and transaction records. It's responsible for updating stock levels, checking reorder rules, and confirming stock levels. The InventoryCommand class acts as the controller by handling user input and coordinating the workflow with InformationService. This reduces coupling and improves maintainability.

The processLabOrder use case design adopts the Repository Pattern and Service Layer for clear separation of concerns. The ProcessLabOrderCommand acts as the controller, delegating business logic to the LabOrderService. The service coordinates between the LabOrderRepository and PatientFileManager. The LabOrder model follows the State Patterns for status transitions.

Critical result detection is handled by the service layer. This approach ensures high maintainability, low coupling, and adherence to the Open/Closed Principle.

Use Cases and Communication Diagrams

Aidan Sousley

Name: orderMedication

Actor: Inventory Manager (IM)

Precondition:

- IM is authenticated
- Medication catalog exists
- Inventory thresholds are defined

Main Success Scenario:

1. IM logs into the hospital ERP system.
2. IM navigates to the inventory management module.
3. IM selects the medication to be reordered.
4. System checks current stock levels against reorder thresholds.
5. IM enters order quantity.
6. IM confirms the order.
7. System records the order and updates "pending orders" status.
8. Supplier receives the order request.
9. System logs the transaction for audit and reporting.

Alternate Flows:

1A. Medication Not Found in Catalog

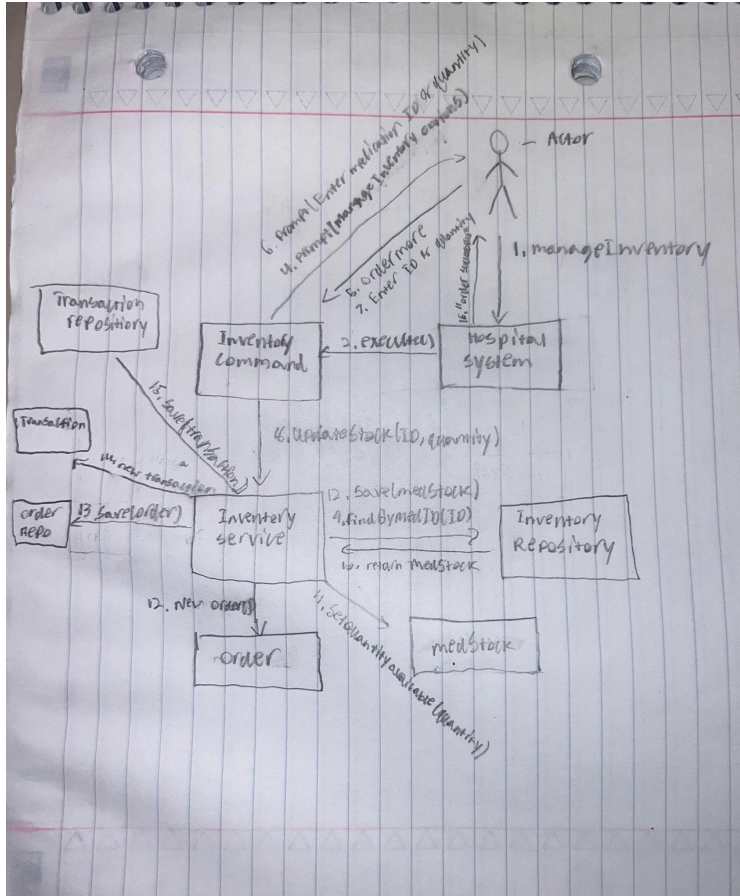
- IM attempts to order a medication not listed.
- System prompts to enter a different medication.

2A. Stock Above Threshold

- System detects sufficient stock.
- IM is notified that reorder is unnecessary.

3A. Invalid Quantity

- IM enters quantity outside allowed limits.
- System rejects and prompts correction.



Harley Peacher

Name: manageInventory

Actor: Inventory Manager (IM)

Precondition:

- IM is authenticated
- Inventory module is available and initialized
- Inventory data exists in the system

Main Success Scenario:

1. IM starts the program and is prompted to enter a command
2. IM enters the manage inventory command.
3. System displays a submenu with options: "view inventory," "withdraw from inventory," and "Add new medication to catalog."
4. IM selects "view inventory."
5. System retrieves and displays inventory levels.
6. IM reviews the data.

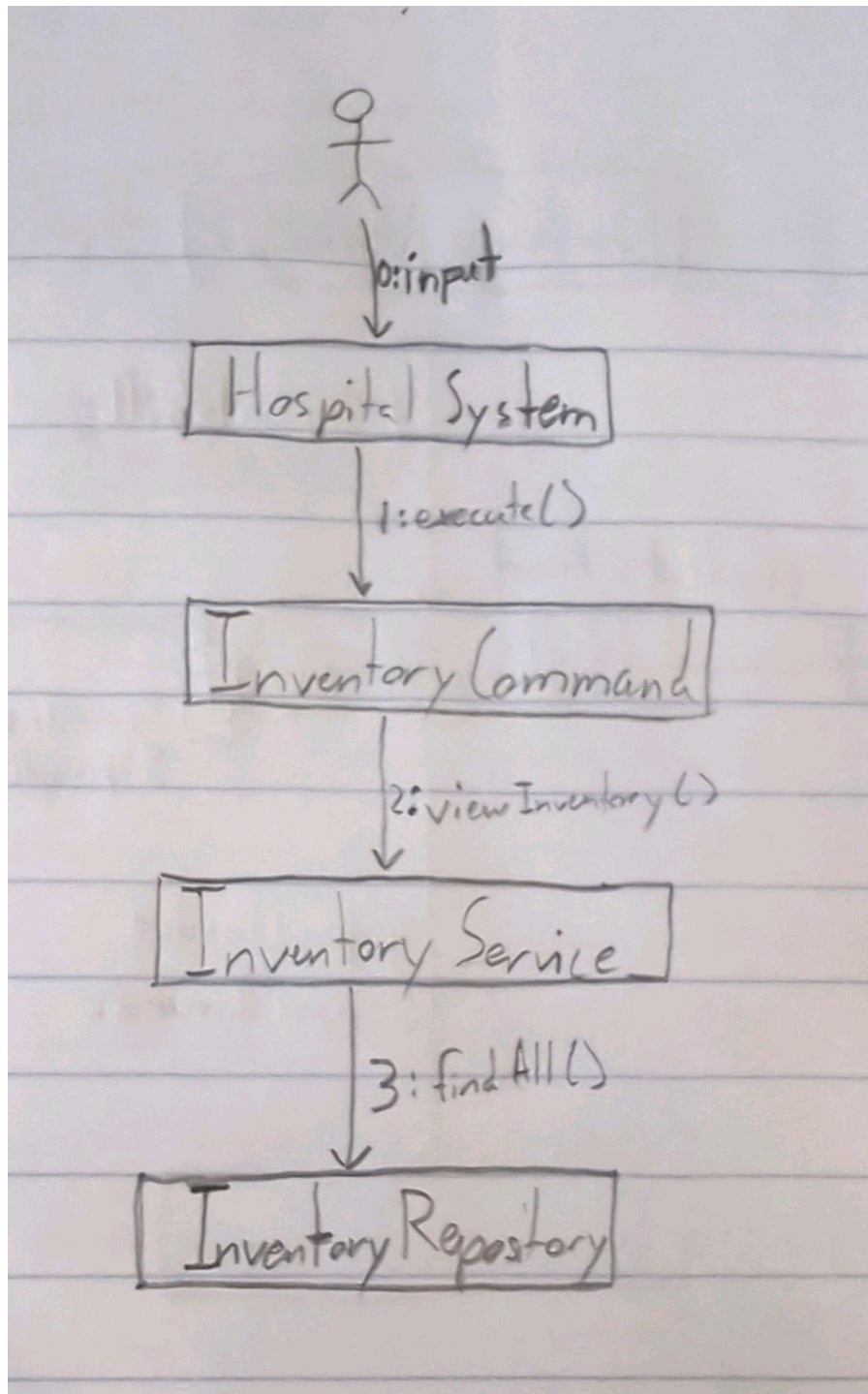
Alternate Flows:

1A. Withdraw From Inventory:

- IM selects "withdraw from inventory."
- System prompts for medication id and quantity.
- IM enters details.
- System validates stock availability.
- If sufficient stock exists, system deducts quantity and updates records.

2A. Insufficient Stock:

- System detects requested quantity exceed availability.
- System rejects withdrawal and notifies IM.



Tirmidi Mohamed

Name: processLabOrder

Actor: Doctor, Lab Technician

Precondition:

- Patient is admitted or has an active visit record.
- The required test/service exists in the system catalogue.

Main Success Scenario::

1. Doctor selects a patient's active visit record and uses "Order Test" to specify the required test(s).
2. System creates a new Lab Order record with status "Pending Lab Collection."
3. Lab Technician views pending orders and enters the test results (data and notes).
4. System sets the status to "Results Ready for Review."
5. Doctor reviews the results and clicks "Approve."
6. System updates the order status to "Finalized" and attaches the approved results to the patient's medical record.

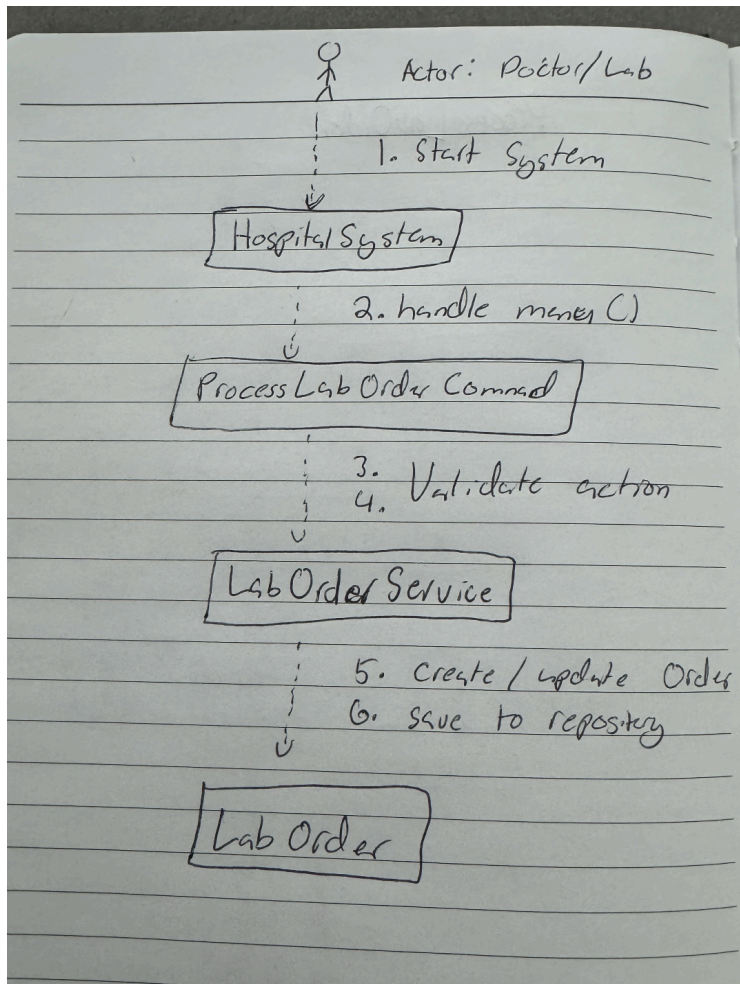
Alternate Flows:

1A. Order Cancellation:

- Doctor cancels the order before results are entered.
- System archives the order as "Canceled."

2A. Abnormal Results:

- System detects results outside predefined critical ranges.
- System flags the result as "Critical" and sends an urgent notification to the ordering Doctor.



Josh Slick

Name: generateHospitalUtilizationReport

Actor: Hospital Administrator, Hospital System, Hospital, DoctorManager, EmployeeViewer, PatientFileManager

Precondition:

- The Hospital System is running and the user is at the main command menu.
- Initial staff (doctors and nurses) have been loaded into the system (addStuff()) and/or persisted data).
- Admitted patients and their records are available in memory and/or retrievable via PatientFileManager.
- Appointments are stored as part of each patient's PatientRecord.

Main Success Scenario::

1. The actor selects the "Generate Hospital Utilization Report" command from the main menu.
2. The system gathers hospital-level metrics, including total capacity and the number of currently admitted patients.
3. The system calculates occupancy statistics.
4. The system retrieves nurse workload information
5. The system identifies workload patterns
6. The system retrieves doctor information and appointment data
7. The system highlights doctors with unusually high appointment counts.
8. The system composes a summary report
9. The system displays the utilization report in a clear and organized format.
10. The actor reviews the report and may adjust staffing, reassign patients, or change scheduling policies outside the system.

Alternate Flows:

A1 – No patients currently admitted:

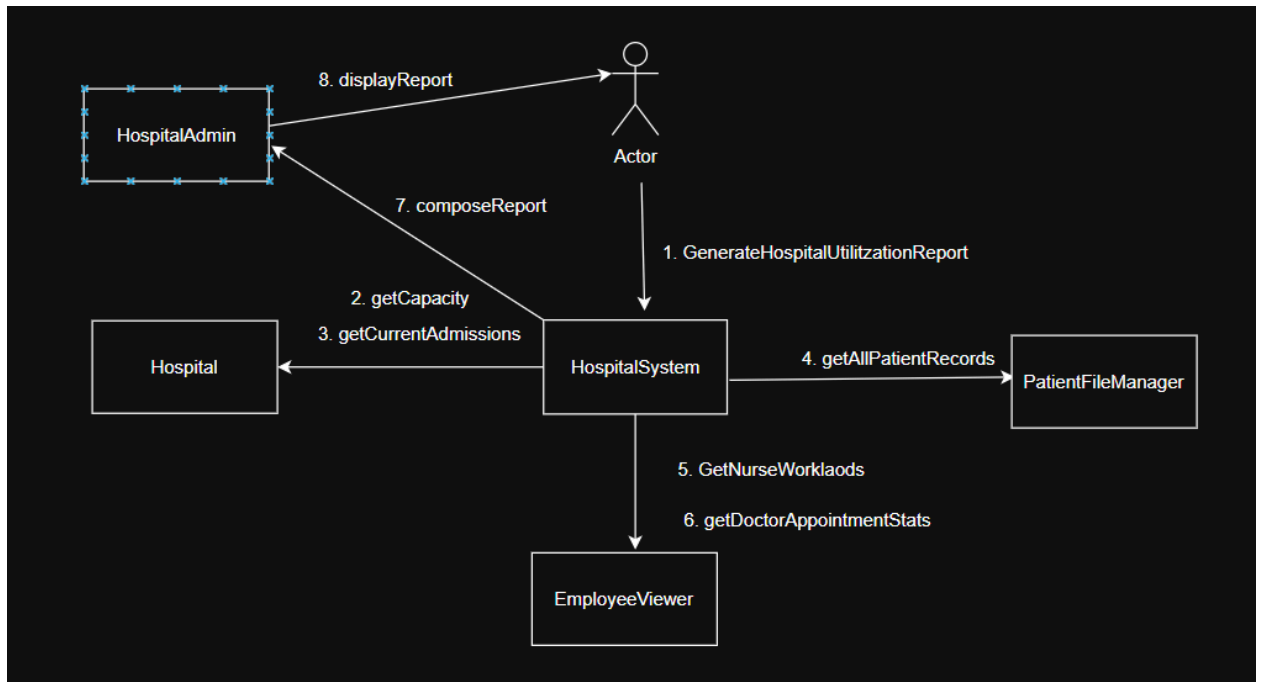
- System reports: "No patients are currently admitted. Occupancy is 0%.
- Nurse and doctor workload sections may still be shown but with zero active patient/appointment counts.
- Use case continues to completion with reduced data.

A2 – No appointments scheduled

- System includes a note in the report: "No appointments are currently scheduled.
- Doctor workload table is either omitted or shown with zero counts for all doctors.

A3 – Missing or non-accessible patient records

- System prints an error message such as: "Warning: Some patient records could not be loaded and may not be reflected in this report."
- System proceeds using only successfully loaded data.
- Use case completes with a partial but still useful report.



Cayden Olsen

Name: createIncidentReport

Actor: Nurse, Security Staff, Administrator (Reporter), Incident Manager, Supervisor

Precondition:

- Reporter is authenticated in the system (or working at an authenticated terminal).
- Incident reporting module is available.
- Reporter has sufficient privileges to submit incident reports (or an anonymous report option exists if configured).

Main Success Scenario:

1. Reporter selects "Report Incident" from the system menu or mobile interface.
2. System displays the Incident Report form (fields: reporter, date/time, location, incident type, severity, involved patients/staff, description, attachments/photos, witnesses).
3. Reporter fills out required fields and optionally attaches photos or documents.
4. Reporter clicks "Submit".
5. System validates required fields and verifies referenced patient/staff IDs (if provided).
6. System creates a new IncidentReport record in IncidentRepository with status REPORTED and generates a unique Incident ID and timestamp.
7. System sends an automatic notification to the configured Supervisor and IncidentManager (email/alert in-app) including a short summary and link to the full report.
8. System optionally creates follow-up tasks (e.g., assign maintenance, schedule review meeting) and links them to the incident record.
9. System logs the action in the audit trail for compliance.
10. System displays a confirmation to the Reporter with the Incident ID and expected next steps.

Alternate Flows:

A1 – Missing Required Fields:

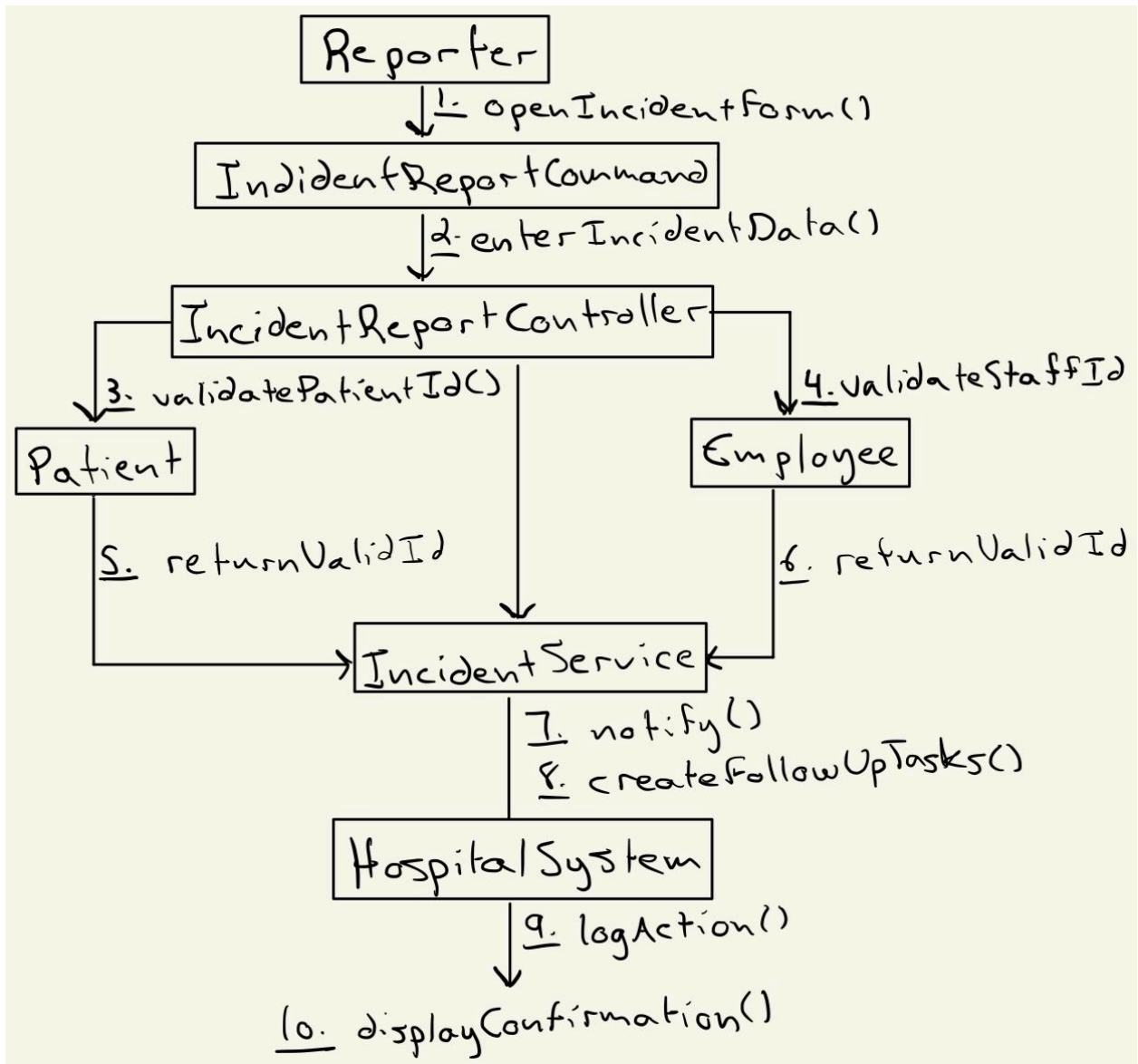
- At step 5, validation fails because required fields are missing.
- System highlights missing fields and prompts the Reporter to complete them.
- Reporter fills missing fields and re-submits; continue from step 6.

A2 – Invalid Patient/Staff IDs:

- At step 5, referenced patient or staff IDs are not found.
- System prompts the Reporter to correct or omit the IDs.
- Reporter corrects IDs or proceeds without linking; continue from step 6.

A4 – Escalation Required (Critical Incident):

- At step 7, if severity meets predefined "critical" thresholds (e.g., patient harm), system automatically triggers an emergency escalation workflow: immediate push alerts to on-duty leadership, creates high-priority tasks, and records response timestamps.
- Incident status is set to ESCALATED and tracked separately.



Interface List

public interface ILabOrderRepository {

```
    void save(LabOrder order);  
    void update(LabOrder order);  
    LabOrder findById(String orderId);  
    List<LabOrder> findById(String patientId);  
    List<LabOrder> findAllPending();  
    List<LabOrder> findAll();
```

```
}
```

public interface ILabOrderService {

```
    LabOrder createOrder(String patientId, String doctorId, String testName);  
    boolean submitResults(String orderId, String results, String notes);  
    boolean approveOrder(String orderId);  
    LabOrder getOrderById(String orderId);  
    List<LabOrder> getPendingOrders();  
    List<LabOrder> getPendingOrdersForReview();  
    boolean cancelOrder(String orderId);  
    boolean validateCriticalResults(String results);
```

```
}
```


Iteration Three Software

```

package com.hospital.service;

import com.hospital.model.Appointment;
import com.hospital.model.Doctor;
import com.hospital.model.DoctorWorkloadEntry;
import com.hospital.model.Hospital;
import com.hospital.model.Nurse;
import com.hospital.model.NurseWorkloadEntry;
import com.hospital.model.PatientRecord;
import com.hospital.model.UtilizationReport;
import com.hospital.repository.impl.PatientFileManager;

import java.util.ArrayList;
import java.util.List;

public class UtilizationReportService {

    private final Hospital hospital;
    private final DoctorManager doctorManager;

    public UtilizationReportService(Hospital hospital, DoctorManager doctorManager) {
        this.hospital = hospital;
        this.doctorManager = doctorManager;
    }

    public UtilizationReport generateReport(PatientFileManager fileManager) {

        List<String> warnings = new ArrayList<>();
        List<String> notes = new ArrayList<>();

        // ----- Hospital-level metrics -----
        int totalCapacity = hospital.getCapacity();
        List<String> admittedIds = hospital.getPatients();
        int currentOccupancy = admittedIds.size();
        double occupancyRate = (totalCapacity == 0)
            ? 0.0
            : (currentOccupancy * 100.0) / totalCapacity;

        if (currentOccupancy == 0) {
            notes.add("No patients are currently admitted. Occupancy is 0%.");
        }

        // ----- Load patient records for admitted patients (for nurse workload + warnings) -----
        List<PatientRecord> admittedRecords = new ArrayList<>();

```

```

List<String> missingIds = new ArrayList<>();

for (String pid : admittedIds) {
    try {
        PatientRecord record = fileManager.getPatientRecord(pid);
        if (record != null) {
            admittedRecords.add(record);
        } else {
            missingIds.add(pid);
        }
    } catch (Exception e) {
        missingIds.add(pid);
    }
}

if (!missingIds.isEmpty()) {
    warnings.add("Warning: Missing patient records: " + String.join(", ", missingIds));
}

// ----- Nurse workload (based on admitted patients) -----
List<NurseWorkloadEntry> nurseWorkloads = new ArrayList<>();
for (Nurse nurse : hospital.getNurses()) {
    int count = hospital.getPatientsAssignedTo(nurse.getId()).size();
    nurseWorkloads.add(new NurseWorkloadEntry(nurse, count));
}

List<DoctorWorkloadEntry> doctorWorkloads = new ArrayList<>();

List<PatientRecord> allRecords = new ArrayList<>();
try {
    allRecords = PatientFileManager.getAllPatientRecords();
} catch (Exception e) {
    warnings.add("Warning: Could not load all patient records for doctor workload.");
}

int smithCount = 0;
int johnsonCount = 0;
int williamsCount = 0;

for (PatientRecord pr : allRecords) {
    if (pr == null || pr.getAppointments() == null) {
        continue;
    }
}

```

```

    }

    for (Appointment appt : pr.getAppointments()) {
        if (appt == null || appt.getDoctorName() == null) {
            continue;
        }

        String name = appt.getDoctorName().toLowerCase();

        if (name.contains("smith")) {
            smithCount++;
        } else if (name.contains("johnson")) {
            johnsonCount++;
        } else if (name.contains("williams")) {
            williamsCount++;
        }
    }
}

int globalAppts = smithCount + johnsonCount + williamsCount;

for (Doctor doctor : doctorManager.getAllDoctors()) {
    String doctorNameLower = doctor.getName().toLowerCase();
    int count = 0;

    if (doctorNameLower.contains("smith")) {
        count = smithCount;
    } else if (doctorNameLower.contains("johnson")) {
        count = johnsonCount;
    } else if (doctorNameLower.contains("williams")) {
        count = williamsCount;
    }

    boolean overloaded = count > 20;
    doctorWorkloads.add(new DoctorWorkloadEntry(doctor, count, overloaded));
}

if (globalAppts == 0) {
    notes.add("No appointments are currently scheduled.");
}

//
return new UtilizationReport(

```

```

        totalCapacity,
        currentOccupancy,
        occupancyRate,
        nurseWorkloads,
        doctorWorkloads,
        warnings,
        notes
    );
}
}

```

```
package com.hospital.model;
```

```
public class DoctorWorkloadEntry {
```

```
    private final Doctor doctor;
    private final int appointmentCount;
    private final boolean overloaded;
```

```
    public DoctorWorkloadEntry(Doctor doctor, int appointmentCount, boolean overloaded) {
        this.doctor = doctor;
        this.appointmentCount = appointmentCount;
        this.overloaded = overloaded;
    }
```

```
    public Doctor getDoctor()      { return doctor; }
    public int getAppointmentCount() { return appointmentCount; }
    public boolean isOverloaded()   { return overloaded; }
}
```

```
package com.hospital.model;
```

```
public class NurseWorkloadEntry {
```

```
    private final Nurse nurse;
    private final int activePatientCount;
```

```
    public NurseWorkloadEntry(Nurse nurse, int activePatientCount) {
        this.nurse = nurse;
        this.activePatientCount = activePatientCount;
    }
```

```
    public Nurse getNurse()      { return nurse; }
    public int getActivePatientCount(){ return activePatientCount; }
}
```

```

package com.hospital.model;

import java.util.List;

public class UtilizationReport {

    private final int totalCapacity;
    private final int currentOccupancy;
    private final double occupancyRate;

    private final List<NurseWorkloadEntry> nurseWorkloads;
    private final List<DoctorWorkloadEntry> doctorWorkloads;

    private final List<String> warnings;
    private final List<String> notes;

    public UtilizationReport(int totalCapacity,
                             int currentOccupancy,
                             double occupancyRate,
                             List<NurseWorkloadEntry> nurseWorkloads,
                             List<DoctorWorkloadEntry> doctorWorkloads,
                             List<String> warnings,
                             List<String> notes) {
        this.totalCapacity = totalCapacity;
        this.currentOccupancy = currentOccupancy;
        this.occupancyRate = occupancyRate;
        this.nurseWorkloads = nurseWorkloads;
        this.doctorWorkloads = doctorWorkloads;
        this.warnings = warnings;
        this.notes = notes;
    }

    public int getTotalCapacity()    { return totalCapacity; }
    public int getCurrentOccupancy() { return currentOccupancy; }
    public double getOccupancyRate() { return occupancyRate; }
    public List<NurseWorkloadEntry> getNurseWorkloads() { return nurseWorkloads; }
    public List<DoctorWorkloadEntry> getDoctorWorkloads() { return doctorWorkloads; }
    public List<String> getWarnings() { return warnings; }
    public List<String> getNotes()    { return notes; }
}

package com.hospital.command.impl;

import com.hospital.command.ICommand;

```

```

import com.hospital.model.Doctor;
import com.hospital.model.DoctorWorkloadEntry;
import com.hospital.model.Nurse;
import com.hospital.model.NurseWorkloadEntry;
import com.hospital.model.UtilizationReport;
import com.hospital.repository.impl.PatientFileManager;
import com.hospital.service.UtilizationReportService;

import java.util.Scanner;

public class UtilizationReportCommand implements ICommand {

    private final UtilizationReportService service;
    private final PatientFileManager patientFileManager;
    @SuppressWarnings("unused")
    private final Scanner sc;

    public UtilizationReportCommand(UtilizationReportService service,
                                    PatientFileManager fileManager,
                                    Scanner sc) {
        this.service = service;
        this.patientFileManager = fileManager;
        this.sc = sc;
    }

    @Override
    public String getToken() {

        return "generate_hospital_report";
    }

    @Override
    public void execute() {
        UtilizationReport report = service.generateReport(patientFileManager);
        print(report);
    }

    private void print(UtilizationReport r) {
        System.out.println("==== Hospital Utilization Report =====");
        System.out.printf("Capacity: %d%n", r.getTotalCapacity());
        System.out.printf("Admitted: %d%n", r.getCurrentOccupancy());
        System.out.printf("Occupancy: %.2f%%\n", r.getOccupancyRate());

        System.out.println("\n-- Nurse Workload --");
    }
}

```

```

    for (NurseWorkloadEntry n : r.getNurseWorkloads()) {
        Nurse nurse = n.getNurse();
        System.out.printf("%s -> %d patients%n",
            nurse.getName(), n.getActivePatientCount());
    }

    System.out.println("\n-- Doctor Workload --");
    for (DoctorWorkloadEntry d : r.getDoctorWorkloads()) {
        Doctor doc = d.getDoctor();
        String overload = d.isOverloaded() ? " (HIGH)" : "";
        System.out.printf("%s -> %d appointments%s%n",
            doc.getName(), d.getAppointmentCount(), overload);
    }

    if (!r.getWarnings().isEmpty()) {
        System.out.println("\nWarnings:");
        r.getWarnings().forEach(w -> System.out.println(" - " + w));
    }

    if (!r.getNotes().isEmpty()) {
        System.out.println("\nNotes:");
        r.getNotes().forEach(n -> System.out.println(" - " + n));
    }

    System.out.println("=====\n");
}
}

```

```

public class InventoryCommand implements ICommand {
    private Scanner sc;
    private InventoryService invService;

    public InventoryCommand(Scanner scanner, InventoryService inventoryService) {
        this.sc = scanner;
        this.invService = inventoryService;
    }

    @Override
    public String getToken() {
        return "manage_inventory";
    }

    @Override
    public void execute() {

```



```

while (true) {
    System.out.println("\n=== Inventory Manager ===");
    System.out.println("1. View Inventory");
    System.out.println("2. Order More");
    System.out.println("3. Withdraw from Inventory");
    System.out.println("4. Back to Main Menu");
    System.out.print("Select option: ");

    String choice = sc.nextLine().trim();
    System.out.println();

    switch (choice) {
        case "1":
            viewInventory();
            break;
        case "2":
            orderMore();
            break;
        case "3":
            withdrawFromInventory();
            break;
        case "4":
            return;
        default:
            System.out.println("Invalid option. Please try again.");
    }
}

private void viewInventory() {
    List<MedicationStock> stock = invService.viewInventory();
    if (stock == null || stock.isEmpty()) {
        System.out.println("Inventory is empty");
        return;
    }
    for (MedicationStock s : stock) {
        System.out.println(s);
    }
}

private void orderMore() {
    System.out.println("Enter medication id you'd like to order: ");
    String id = sc.nextLine().trim();
    System.out.println("Enter quantity: ");
}

```

```

        int quantity = sc.nextInt();
        sc.nextLine();

        try {
            invService.updateStock(id, quantity);
            System.out.println("Successfully ordered " + quantity + " of medicine " + id);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    private void withdrawFromInventory() {
        System.out.println("Enter medication id: ");
        String id = sc.nextLine().trim();
        System.out.println("Enter quantity: ");
        int quantity = sc.nextInt();
        sc.nextLine();

        try {
            invService.withdrawMedication(id, quantity);
            System.out.println("Successfully withdrew " + quantity + " of medicine " + id);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}

```

public class InventoryService {

```

    private final InventoryRepository inventoryRepository;
    private final TransactionRepository transactionRepository;
    private final MedicationOrderRepository medicationOrderRepository;

    public InventoryService(InventoryRepository inventoryRepository,
        TransactionRepository transactionRepository, MedicationOrderRepository
medicationOrderRepository) {
        this.inventoryRepository = inventoryRepository;
        this.transactionRepository = transactionRepository;
        this.medicationOrderRepository = medicationOrderRepository;
    }

    public List<MedicationStock> viewInventory() {
        return inventoryRepository.findAll();
    }
}

```

```

public void withdrawMedication(String medicationId, int quantity) {
    Optional<MedicationStock> stockOpt =
inventoryRepository.findByMedicationId(medicationId);

    if (stockOpt.isEmpty()) {
        throw new IllegalArgumentException("Medication not found in inventory");
    }

    MedicationStock stock = stockOpt.get();

    if (stock.getQuantityAvailable() < quantity) {
        throw new IllegalStateException("Insufficient stock for withdrawal");
    }

    // Deduct stock
    stock.setQuantityAvailable(stock.getQuantityAvailable() - quantity);
    inventoryRepository.save(stock);

    // Log transaction
    Transaction tx = new Transaction("WITHDRAWAL", medicationId, quantity);
    transactionRepository.save(tx);
}

public void updateStock(String medicationId, int quantityAdded) {
    Optional<MedicationStock> stockOpt =
inventoryRepository.findByMedicationId(medicationId);
    if (stockOpt.isEmpty()) {
        throw new IllegalArgumentException("Medication not found in inventory.");
    }
    if (!isBelowThreshold(medicationId)) {
        throw new IllegalArgumentException("Reorder threshold not reached.");
    }

    MedicationStock stock = stockOpt.get();
    stock.setQuantityAvailable(stock.getQuantityAvailable() + quantityAdded);
    inventoryRepository.save(stock);

    MedicationOrder order = new MedicationOrder(medicationId, quantityAdded);
    medicationOrderRepository.save(order);

    Transaction tx = new Transaction("RESTOCK", medicationId, quantityAdded);
    transactionRepository.save(tx);
}

```

```

    }

    public void addMedication(String medicationId, String name, int initialQuantity, int
reorderThreshold) {
        if (inventoryRepository.findByMedicationId(medicationId).isPresent()) {
            throw new IllegalStateException("Medication already exists in inventory");
        }

        MedicationStock stock = new MedicationStock(medicationId, name, initialQuantity,
reorderThreshold);
        inventoryRepository.save(stock);

        Transaction tx = new Transaction("ADD_MEDICATION", medicationId, initialQuantity);
        transactionRepository.save(tx);
    }

    public boolean isBelowThreshold(String medicationId) {
        Optional<MedicationStock> stockOpt =
inventoryRepository.findByMedicationId(medicationId);
        return stockOpt.map(stock -> stock.getQuantityAvailable() < stock.getReorderThreshold())
            .orElse(false);
    }
}

public class InventoryRepository {
    private final File inventoryDir;

    public InventoryRepository(String basePath) {
        this.inventoryDir = new File(basePath, "inventory");
        if (!inventoryDir.exists()) {
            inventoryDir.mkdirs();
        }
    }

    public void save(MedicationStock stock) {
        File file = new File(inventoryDir, stock.getMedicationId() + ".json");
        JsonSerializer.writeToFile(file, stock);
    }

    public Optional<MedicationStock> findByMedicationId(String medicationId) {

        File file = new File(inventoryDir, medicationId + ".json");

        if (!file.exists())

```

```

        return Optional.empty();
    }
    return Optional.of(JsonSerializer.readFromFile(file, MedicationStock.class));
}

public List<MedicationStock> findAll() {
    List<MedicationStock> stocks = new ArrayList<>();
    File[] files = inventoryDir.listFiles((dir, name) -> name.endsWith(".json"));

    if (files != null) {
        for (File file : files) {
            try {
                MedicationStock stock = JsonSerializer.readFromFile(file, MedicationStock.class);
                stocks.add(stock);
            } catch (RuntimeException e) {
                System.err.println("Skipping invalid file: " + file.getName());
            }
        }
    }
    return stocks;
}
}

```

```

public class MedicationOrderRepository {
    private final File orderDir;

    public MedicationOrderRepository(String basePath) {
        this.orderDir = new File(basePath, "orders");
        if (!orderDir.exists()) {
            orderDir.mkdirs();
        }
    }

    public void save(MedicationOrder mo) {
        File file = new File(orderDir, "ORD" + mo.getOrderId() + ".json");
        JsonSerializer.writeToFile(file, mo);
    }

    public MedicationOrder findById(String transactionId) {
        File file = new File(orderDir, transactionId + ".json");
        if (!file.exists())
            return null;
        return JsonSerializer.readFromFile(file, MedicationOrder.class);
    }
}

```

```

public class TransactionRepository {
    private final File transactionDir;

    public TransactionRepository(String basePath) {
        this.transactionDir = new File(basePath, "transactions");
        if (!transactionDir.exists()) {
            transactionDir.mkdirs();
        }
    }

    public void save(Transaction tx) {
        File file = new File(transactionDir, tx.getTransactionId() + ".json");
        JsonSerializer.writeToFile(file, tx);
    }

    public Transaction findById(String transactionId) {
        File file = new File(transactionDir, transactionId + ".json");
        if (!file.exists())
            return null;
        return JsonSerializer.readFromFile(file, Transaction.class);
    }
}

```

```

public class Transaction {
    private String transactionId;
    private String type; // e.g., "WITHDRAWAL", "RESTOCK"
    private String medicationId;
    private int quantity;
    private LocalDateTime timestamp;

    public Transaction() {
    }

    public Transaction(String type, String medicationId, int quantity) {
        this.transactionId = UUID.randomUUID().toString();
        this.type = type;
        this.medicationId = medicationId;
        this.quantity = quantity;
        this.timestamp = LocalDateTime.now();
    }

    // Getters and Setters
    public String getTransactionId() {

```

```

        return transactionId;
    }

    public void setTransactionId(String transactionId) {
        this.transactionId = transactionId;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public String getMedicationId() {
        return medicationId;
    }

    public void setMedicationId(String medicationId) {
        this.medicationId = medicationId;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }

    @Override
    public String toString() {
        return "Transaction{" +
            "transactionId=\"" + transactionId + "\" +
            ", type=\"" + type + "\" +

```

```

        ", medicationId=" + medicationId + "\" +
        ", quantity=" + quantity +
        ", timestamp=" + timestamp +
        '>';
    }

```

public class MedicationOrder {

```

    private String medicationId;
    private int quantity;
    // private LocalDateTime orderDate;
    private int orderId;
    private static int orderIdCounter = 0;
    private String status; // e.g., "PENDING", "FULFILLED"
    public MedicationOrder() {
    }

    public MedicationOrder(String medicationId, int quantity) {
        this.medicationId = medicationId;
        this.quantity = quantity;
    //    this.orderDate = LocalDateTime.now();
        this.orderId = ++orderIdCounter;
        this.status = "PENDING";
    }

    public String getMedicationId() {
        return medicationId;
    }

    public void setMedicationId(String medicationId) {
        this.medicationId = medicationId;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
    /* public LocalDateTime getOrderDate() {
        return orderDate;
    } */
    public int getOrderId() {

```



```

        return orderId;
    }
    public String getStatus() {
        return status;
    }
    public void setStatus(String status) {
        this.status = status;
    }
}

```

public class MedicationStock {

```

    private String medicationId;
    private String name;
    private int quantityAvailable;
    private int reorderThreshold;

```

```

    public MedicationStock() {
    }

```

```

    public MedicationStock(String medicationId, String name, int quantityAvailable, int
reorderThreshold) {
        this.medicationId = medicationId;
        this.name = name;
        this.quantityAvailable = quantityAvailable;
        this.reorderThreshold = reorderThreshold;
    }

```

// Getters and Setters

```

    public String getMedicationId() {
        return medicationId;
    }

```

```

    public void setMedicationId(String medicationId) {
        this.medicationId = medicationId;
    }

```

```

    public String getName() {
        return name;
    }

```

```

    public void setName(String name) {
        this.name = name;
    }

```

```

    public int getQuantityAvailable() {
        return quantityAvailable;
    }

    public void setQuantityAvailable(int quantityAvailable) {
        this.quantityAvailable = quantityAvailable;
    }

    public int getReorderThreshold() {
        return reorderThreshold;
    }

    public void setReorderThreshold(int reorderThreshold) {
        this.reorderThreshold = reorderThreshold;
    }

    @Override
    public String toString() {
        return "MedicationStock{" +
            "medicationId=" + medicationId + "\" +
            ", name=" + name + "\" +
            ", quantityAvailable=" + quantityAvailable +
            ", reorderThreshold=" + reorderThreshold +
            "}";
    }
}

package com.hospital.command.impl;

import com.hospital.command.ICommand;
import com.hospital.service.IncidentReportController;

import java.util.Scanner;

public class CreateIncidentReportCommand implements ICommand {

    private final IncidentReportController controller;
    private final Scanner scanner;

    public CreateIncidentReportCommand(IncidentReportController controller, Scanner scanner)
    {
        this.controller = controller;
        this.scanner = scanner;
    }
}

```

```

@Override
public String getToken() {
    return "createIncidentReport";
}

@Override
public void execute() {
    System.out.println("\n=== Create Incident Report ===");

    System.out.print("Enter Patient ID: ");
    String patientId = scanner.nextLine().trim();

    System.out.print("Enter Reporter Employee ID: ");
    String reporterId = scanner.nextLine().trim();

    System.out.print("Enter Incident Description: ");
    String description = scanner.nextLine().trim();

    try {
        boolean success = controller.createIncidentReport(patientId, reporterId, description);

        if (success) {
            System.out.println("\nIncident report successfully created.");
        } else {
            System.out.println("\nERROR: Failed to create incident report.");
        }
    } catch (Exception e) {
        System.out.println("\nERROR: " + e.getMessage());
    }

    System.out.println("=====\n");
}
}

```

```
package com.hospital.model;
```

```
public class IncidentReport {
```

```

    private String id;
    private String patientId;
    private String reporterId;
    private String description;
    private String timestamp;

```

```

public IncidentReport() {}

public IncidentReport(String id, String patientId, String reporterId,
                      String description, String timestamp) {
    this.id = id;
    this.patientId = patientId;
    this.reporterId = reporterId;
    this.description = description;
    this.timestamp = timestamp;
}

public String getId() {
    return id;
}

public void setId(String id) {
    this.id = id;
}

public String getPatientId() {
    return patientId;
}

public void setPatientId(String patientId) {
    this.patientId = patientId;
}

public String getReporterId() {
    return reporterId;
}

public void setReporterId(String reporterId) {
    this.reporterId = reporterId;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

```

```

    public String getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(String timestamp) {
        this.timestamp = timestamp;
    }
}

package com.hospital.repository.impl;

import com.hospital.model.IncidentReport;
import com.hospital.repository.JsonSerializer;
import com.google.gson.reflect.TypeToken;

import java.io.File;
import java.lang.reflect.Type;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

public class IncidentReportRepository {

    private final File storageFile;
    private final Type listType = new TypeToken<List<IncidentReport>>().getType();

    private List<IncidentReport> reports;

    public IncidentReportRepository(String filePath) {
        this.storageFile = new File(filePath);
        this.reports = loadAll();
    }

    private List<IncidentReport> loadAll() {
        if (!storageFile.exists()) {
            return new ArrayList<>();
        }
        return JsonSerializer.readListFromFile(storageFile, listType);
    }

    private void saveAll() {
        JsonSerializer.writeToFile(storageFile, reports);
    }
}

```

```

public String save(IncidentReport report) {
    if (report.getId() == null || report.getId().isEmpty()) {
        report.setId(UUID.randomUUID().toString());
        reports.add(report);
    } else {
        reports.removeIf(r -> r.getId().equals(report.getId()));
        reports.add(report);
    }
    saveAll();
    return report.getId();
}

public IncidentReport findById(String id) {
    return reports.stream()
        .filter(r -> r.getId().equals(id))
        .findFirst()
        .orElse(null);
}

public List<IncidentReport> findAll() {
    return new ArrayList<>(reports);
}
}

package com.hospital.service;

import com.hospital.service.IncidentReportService;

public class IncidentReportController {

    private final IncidentReportService service;

    public IncidentReportController(IncidentReportService service) {
        this.service = service;
    }

    public boolean createIncidentReport(String patientId, String reporterId, String description) {
        return service.createIncident(patientId, reporterId, description);
    }
}

package com.hospital.service;

import com.hospital.model.Employee;

```

```

import com.hospital.model.IncidentReport;
import com.hospital.model.PatientRecord;
import com.hospital.repository.impl.IncidentReportRepository;
import com.hospital.repository.impl.PatientFileManager;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class IncidentReportService {

    private final IncidentReportRepository repo;
    private final PatientFileManager patientFiles;

    public IncidentReportService(
        IncidentReportRepository repo,
        PatientFileManager patientFiles) {

        this.repo = repo;
        this.patientFiles = patientFiles;
    }

    public boolean createIncident(String patientId, String reporterId, String description) {

        PatientRecord patientRecord = patientFiles.getPatientRecord(patientId);
        if (patientRecord == null) {
            throw new IllegalArgumentException("Invalid patient ID.");
        }

        Employee reporter = new Employee();
        reporter.setEmployeeId(reporterId);

        if (reporter.getEmployeeId() == null || reporter.getEmployeeId().isEmpty()) {
            throw new IllegalArgumentException("Invalid reporter ID.");
        }

        String timestamp = LocalDateTime.now()
            .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));

        IncidentReport report = new IncidentReport(
            null,
            patientId,
            reporter.getEmployeeId(),
            description,
            timestamp

```

```

    );

    String id = repo.save(report);
    return (id != null);
}

public IncidentReport getIncident(String id) {
    return repo.findById(id);
}
}

// ProcessLabOrder Use Case - Tirmidi Mohamed

```

```

package com.hospital.model;

```

```

import java.text.SimpleDateFormat;
import java.util.Date;

```

```

public class LabOrder {

```

```

    private String orderId;
    private String patientId;
    private String testName;
    private String orderingDoctorId;
    private String status;
    private String results;
    private String technicianNotes;
    private String dateOrdered;
    private String dateCompleted;
    private boolean isCritical;

```

```

    public LabOrder(String orderId, String patientId, String testName, String orderingDoctorId) {
        this.orderId = orderId;
        this.patientId = patientId;
        this.testName = testName;
        this.orderingDoctorId = orderingDoctorId;
        this.status = "Pending Lab Collection";
        this.results = null;
        this.technicianNotes = null;
        this.dateOrdered = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new Date());
        this.dateCompleted = null;
        this.isCritical = false;
    }

```

```

// Getters and setters

```



```

public String getOrderId() { return orderId; }
public void setOrderId(String orderId) { this.orderId = orderId; }
public String getPatientId() { return patientId; }
public String getTestName() { return testName; }
public String getOrderingDoctorId() { return orderingDoctorId; }
public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }
public String getResults() { return results; }
public void setResults(String results, String notes) {
    this.results = results;
    this.technicianNotes = notes;
}
public String getTechnicianNotes() { return technicianNotes; }
public String getDateOrdered() { return dateOrdered; }
public String getDateCompleted() { return dateCompleted; }
public void setDateCompleted(String dateCompleted) { this.dateCompleted = dateCompleted;
}
    public boolean isCritical() { return isCritical; }
    public void markCritical() { this.isCritical = true; }
    public boolean isFinalized() { return "Finalized".equals(status); }
}

```

```
package com.hospital.repository.impl;
```

```

import com.hospital.model.LabOrder;
import com.hospital.repository.ILabOrderRepository;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

```

```
public class LabOrderRepository implements ILabOrderRepository {
```

```

    private List<LabOrder> orders;
    private static int orderIdCounter = 0;

```

```

    public LabOrderRepository() {
        this.orders = new ArrayList<>();
    }

```

```
@Override
```

```

    public void save(LabOrder order) {
        if (order.getOrderId() == null || order.getOrderId().isEmpty()) {
            order.setOrderId("LO" + (++orderIdCounter));
        }
        orders.add(order);
    }

```

```

    }

    @Override
    public void update(LabOrder order) {
        for (int i = 0; i < orders.size(); i++) {
            if (orders.get(i).getOrderId().equals(order.getOrderId())) {
                orders.set(i, order);
                return;
            }
        }
    }

    @Override
    public LabOrder findById(String orderId) {
        for (LabOrder order : orders) {
            if (order.getOrderId().equals(orderId)) {
                return order;
            }
        }
        return null;
    }

    @Override
    public List<LabOrder> findByPatientId(String patientId) {
        return orders.stream()
            .filter(order -> order.getPatientId().equals(patientId))
            .collect(Collectors.toList());
    }

    @Override
    public List<LabOrder> findAllPending() {
        return orders.stream()
            .filter(order -> "Pending Lab Collection".equals(order.getStatus()) ||
                "Results Ready for Review".equals(order.getStatus()))
            .collect(Collectors.toList());
    }

    @Override
    public List<LabOrder> findAll() {
        return new ArrayList<>(orders);
    }
}

package com.hospital.service;

```

```

import com.hospital.model.LabOrder;
import com.hospital.model.PatientRecord;
import com.hospital.model.VisitRecord;
import com.hospital.repository.ILabOrderRepository;
import com.hospital.repository.IPatientFileManager;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.stream.Collectors;

public class LabOrderService implements ILabOrderService {
    private ILabOrderRepository repository;
    private IPatientFileManager patientFileManager;

    public LabOrderService(ILabOrderRepository repository, IPatientFileManager
patientFileManager) {
        this.repository = repository;
        this.patientFileManager = patientFileManager;
    }

    @Override
    public LabOrder createOrder(String patientId, String doctorId, String testName) {
        LabOrder order = new LabOrder(null, patientId, testName, doctorId);
        repository.save(order);
        System.out.println("Lab order created: " + order.getOrderid() + " for test: " + testName);
        return order;
    }

    @Override
    public boolean submitResults(String orderId, String results, String notes) {
        LabOrder order = repository.findById(orderId);
        if (order == null || !"Pending Lab Collection".equals(order.getStatus())) {
            return false;
        }

        order.setResults(results, notes);
        order.setStatus("Results Ready for Review");

        if (validateCriticalResults(results)) {
            order.markCritical();
            System.out.println("WARNING: Critical results detected for order " + orderId);
            System.out.println("URGENT NOTIFICATION sent to Dr. " +
order.getOrderingDoctorId());

```

```

    }

    repository.update(order);
    return true;
}

@Override
public boolean approveOrder(String orderId) {
    LabOrder order = repository.findById(orderId);
    if (order == null || !"Results Ready for Review".equals(order.getStatus())) {
        return false;
    }

    order.setStatus("Finalized");
    order.setDateCompleted(new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new
Date()));
    repository.update(order);

    attachToPatientRecord(order);
    System.out.println("Results attached to patient record: " + order.getPatientId());
    return true;
}

@Override
public LabOrder getOrderById(String orderId) {
    return repository.findById(orderId);
}

@Override
public List<LabOrder> getPendingOrders() {
    return repository.findAllPending().stream()
        .filter(order -> "Pending Lab Collection".equals(order.getStatus()))
        .collect(Collectors.toList());
}

@Override
public List<LabOrder> getPendingOrdersForReview() {
    return repository.findAllPending().stream()
        .filter(order -> "Results Ready for Review".equals(order.getStatus()))
        .collect(Collectors.toList());
}

@Override
public boolean cancelOrder(String orderId) {

```

```

        LabOrder order = repository.findById(orderId);
        if (order == null || order.isFinalized()) {
            return false;
        }
        order.setStatus("Canceled");
        repository.update(order);
        return true;
    }

    @Override
    public boolean validateCriticalResults(String results) {
        if (results == null) return false;
        String lowerResults = results.toLowerCase();
        return lowerResults.contains("critical") ||
            lowerResults.contains("abnormal") ||
            lowerResults.contains("urgent") ||
            lowerResults.contains("high risk") ||
            lowerResults.contains("immediate attention");
    }

    private void attachToPatientRecord(LabOrder order) {
        PatientRecord patientRecord = patientFileManager.getPatientRecord(order.getPatientId());
        if (patientRecord != null && !patientRecord.getVisits().isEmpty()) {
            VisitRecord mostRecentVisit = patientRecord.getMostRecentVisitRecord();
            mostRecentVisit.addLabOrder(order);
            patientFileManager.postPatientRecord(patientRecord);
        }
    }
}

package com.hospital.command.impl;

import com.hospital.command.ICommand;
import com.hospital.model.LabOrder;
import com.hospital.service.ILabOrderService;
import java.util.List;
import java.util.Scanner;

public class ProcessLabOrderCommand implements ICommand {
    private ILabOrderService labOrderService;
    private Scanner scanner;

    public ProcessLabOrderCommand(ILabOrderService labOrderService, Scanner scanner) {
        this.labOrderService = labOrderService;
    }

```

```

        this.scanner = scanner;
    }

    @Override
    public String getToken() {
        return "processlaborder";
    }

    @Override
    public void execute() {
        while (true) {
            System.out.println("\n=== Process Lab Orders ===");
            System.out.println("1. Order Test (Doctor)");
            System.out.println("2. Enter Test Results (Lab Technician)");
            System.out.println("3. Review & Approve Results (Doctor)");
            System.out.println("4. View Pending Orders");
            System.out.println("5. Cancel Order");
            System.out.println("6. Back to Main Menu");
            System.out.print("Select option: ");

            String choice = scanner.nextLine().trim();

            switch (choice) {
                case "1": handleDoctorOrder(); break;
                case "2": handleLabEntry(); break;
                case "3": handleDoctorApproval(); break;
                case "4": viewPendingOrders(); break;
                case "5": cancelOrder(); break;
                case "6": return;
                default: System.out.println("Invalid option. Please try again.");
            }
        }
    }

    private void handleDoctorOrder() {
        System.out.print("Enter Patient ID: ");
        String patientId = scanner.nextLine().trim();
        System.out.print("Enter Doctor ID: ");
        String doctorId = scanner.nextLine().trim();
        System.out.print("Enter Test Name: ");
        String testName = scanner.nextLine().trim();

        LabOrder order = labOrderService.createOrder(patientId, doctorId, testName);
        System.out.println("Success! Lab order created with ID: " + order.getOrderId());
    }

```

```

}

private void handleLabEntry() {
    List<LabOrder> pendingOrders = labOrderService.getPendingOrders();
    if (pendingOrders.isEmpty()) {
        System.out.println("No pending lab orders for result entry.");
        return;
    }

    System.out.println("\nPending Orders:");
    for (LabOrder order : pendingOrders) {
        System.out.println("Order ID: " + order.getId() +
            " | Patient: " + order.getPatientId() +
            " | Test: " + order.getTestName());
    }

    System.out.print("\nEnter Order ID: ");
    String orderId = scanner.nextLine().trim();
    System.out.print("Enter Test Results: ");
    String results = scanner.nextLine().trim();
    System.out.print("Enter Technician Notes (optional): ");
    String notes = scanner.nextLine().trim();

    if (labOrderService.submitResults(orderId, results, notes)) {
        System.out.println("Results successfully submitted!");
    } else {
        System.out.println("Failed to submit results.");
    }
}

private void handleDoctorApproval() {
    List<LabOrder> reviewOrders = labOrderService.getPendingOrdersForReview();
    if (reviewOrders.isEmpty()) {
        System.out.println("No lab orders pending review.");
        return;
    }

    System.out.println("\nOrders Ready for Review:");
    for (LabOrder order : reviewOrders) {
        System.out.println("\nOrder ID: " + order.getId());
        System.out.println("Patient: " + order.getPatientId());
        System.out.println("Test: " + order.getTestName());
        System.out.println("Results: " + order.getResults());
        if (order.isCritical()) {

```

```

        System.out.println("**** CRITICAL RESULTS - REQUIRES IMMEDIATE ATTENTION
****");
    }
}

System.out.print("\nEnter Order ID to approve: ");
String orderId = scanner.nextLine().trim();

if (labOrderService.approveOrder(orderId)) {
    System.out.println("Order approved and finalized!");
} else {
    System.out.println("Failed to approve order.");
}
}

private void viewPendingOrders() {
    List<LabOrder> pendingOrders = labOrderService.getPendingOrders();
    List<LabOrder> reviewOrders = labOrderService.getPendingOrdersForReview();

    if (pendingOrders.isEmpty() && reviewOrders.isEmpty()) {
        System.out.println("No pending orders.");
        return;
    }

    if (!pendingOrders.isEmpty()) {
        System.out.println("\nAwaiting Lab Collection:");
        for (LabOrder order : pendingOrders) {
            System.out.println(order);
        }
    }

    if (!reviewOrders.isEmpty()) {
        System.out.println("\nAwaiting Doctor Review:");
        for (LabOrder order : reviewOrders) {
            System.out.println(order);
        }
    }
}

private void cancelOrder() {
    System.out.print("Enter Order ID to cancel: ");
    String orderId = scanner.nextLine().trim();
    if (labOrderService.cancelOrder(orderId)) {
        System.out.println("Order canceled successfully.");
    }
}

```



```
    } else {  
        System.out.println("Failed to cancel order.");  
    }  
}  
}
```

```

package com.hospital.command.impl;

import java.util.Scanner;
import java.util.List;

import com.hospital.command.ICommand;
import com.hospital.service.InventoryService;
import com.hospital.model.MedicationStock;

public class InventoryCommand implements ICommand {
    private Scanner sc;
    private InventoryService invService;

    public InventoryCommand(Scanner scanner, InventoryService inventoryService) {
        this.sc = scanner;
        this.invService = inventoryService;
    }

    @Override
    public String getToken() {
        return "manage_inventory";
    }

    @Override
    public void execute() {
        while (true) {
            System.out.println("\n=== Inventory Manager ===");
            System.out.println("1. View Inventory");
            System.out.println("2. Order More");
            System.out.println("3. Withdraw from Inventory");
            System.out.println("4. Back to Main Menu");
            System.out.print("Select option: ");

            String choice = sc.nextLine().trim();
            System.out.println();

            switch (choice) {
                case "1":
                    viewInventory();
                    break;
                case "2":
                    orderMore();
                    break;
                case "3":

```

```

        withdrawFromInventory();
        break;
    case "4":
        return;
    default:
        System.out.println("Invalid option. Please try again.");
    }
}
}

private void viewInventory() {
    List<MedicationStock> stock = invService.viewInventory();
    if (stock == null || stock.isEmpty()) {
        System.out.println("Inventory is empty");
        return;
    }
    for (MedicationStock s : stock) {
        System.out.println(s);
    }
}

private void orderMore() {
    System.out.println("Will be finished in another branch");
}

private void withdrawFromInventory() {
    System.out.println("Enter medication id: ");
    String id = sc.nextLine().trim();
    System.out.println("Enter quantity: ");
    int quantity = sc.nextInt();
    sc.nextLine();

    try {
        invService.withdrawMedication(id, quantity);
        System.out.println("Successfully withdrew " + quantity + " of medicine " + id);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
}

```

```
package com.hospital.model;
```

```
public class MedicationStock {
```

```
    private String medicationId;  
    private String name;  
    private int quantityAvailable;  
    private int reorderThreshold;
```

```
    public MedicationStock() {  
    }
```

```
    public MedicationStock(String medicationId, String name, int quantityAvailable, int  
reorderThreshold) {  
        this.medicationId = medicationId;  
        this.name = name;  
        this.quantityAvailable = quantityAvailable;  
        this.reorderThreshold = reorderThreshold;  
    }
```

```
    // Getters and Setters
```

```
    public String getMedicationId() {  
        return medicationId;  
    }
```

```
    public void setMedicationId(String medicationId) {  
        this.medicationId = medicationId;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public void setName(String name) {  
        this.name = name;  
    }
```

```
    public int getQuantityAvailable() {  
        return quantityAvailable;  
    }
```

```
    public void setQuantityAvailable(int quantityAvailable) {  
        this.quantityAvailable = quantityAvailable;  
    }
```

```

public int getReorderThreshold() {
    return reorderThreshold;
}

public void setReorderThreshold(int reorderThreshold) {
    this.reorderThreshold = reorderThreshold;
}

@Override
public String toString() {
    return "MedicationStock{" +
        "medicationId=" + medicationId + "\" +
        ", name=" + name + "\" +
        ", quantityAvailable=" + quantityAvailable +
        ", reorderThreshold=" + reorderThreshold +
        '"';
}
}

```

```

package com.hospital.model;

import java.time.LocalDateTime;
import java.util.UUID;

public class Transaction {
    private String transactionId;
    private String type; // e.g., "WITHDRAWAL", "RESTOCK"
    private String medicationId;
    private int quantity;
    private LocalDateTime timestamp;

    public Transaction() {
    }

    public Transaction(String type, String medicationId, int quantity) {
        this.transactionId = UUID.randomUUID().toString();
        this.type = type;
        this.medicationId = medicationId;
        this.quantity = quantity;
        this.timestamp = LocalDateTime.now();
    }

    // Getters and Setters
    public String getTransactionId() {
        return transactionId;
    }

    public void setTransactionId(String transactionId) {
        this.transactionId = transactionId;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public String getMedicationId() {
        return medicationId;
    }
}

```

```

public void setMedicationId(String medicationId) {
    this.medicationId = medicationId;
}

public int getQuantity() {
    return quantity;
}

public void setQuantity(int quantity) {
    this.quantity = quantity;
}

public LocalDateTime getTimestamp() {
    return timestamp;
}

public void setTimestamp(LocalDateTime timestamp) {
    this.timestamp = timestamp;
}

@Override
public String toString() {
    return "Transaction{" +
        "transactionId=" + transactionId + "\" +
        ", type=" + type + "\" +
        ", medicationId=" + medicationId + "\" +
        ", quantity=" + quantity +
        ", timestamp=" + timestamp +
        '}';
}
}

```

```

package com.hospital.repository.impl;

import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import com.hospital.repository.JsonSerializer;
import com.hospital.model.MedicationStock;

public class InventoryRepository {
    private final File inventoryDir;

    public InventoryRepository(String basePath) {
        this.inventoryDir = new File(basePath, "inventory");
        if (!inventoryDir.exists()) {
            inventoryDir.mkdirs();
        }
    }

    public void save(MedicationStock stock) {
        File file = new File(inventoryDir, stock.getMedicationId() + ".json");
        JsonSerializer.writeToFile(file, stock);
    }

    public Optional<MedicationStock> findById(String medicationId) {
        File file = new File(inventoryDir, medicationId + ".json");
        if (!file.exists())
            return Optional.empty();
        return Optional.of(JsonSerializer.readFromFile(file, MedicationStock.class));
    }

    public List<MedicationStock> findAll() {
        List<MedicationStock> stocks = new ArrayList<>();
        File[] files = inventoryDir.listFiles((dir, name) -> name.endsWith(".json"));

        if (files != null) {
            for (File file : files) {
                try {
                    MedicationStock stock = JsonSerializer.readFromFile(file, MedicationStock.class);
                    stocks.add(stock);
                } catch (RuntimeException e) {
                    System.err.println("Skipping invalid file: " + file.getName());
                }
            }
        }
    }
}

```



```

    }
    }
    return stocks;
}
}

```

```
package com.hospital.repository.impl;
```

```
import java.io.File;
import com.hospital.repository.JsonSerializer;
import com.hospital.model.Transaction;
```

```

public class TransactionRepository {
    private final File transactionDir;

    public TransactionRepository(String basePath) {
        this.transactionDir = new File(basePath, "transactions");
        if (!transactionDir.exists()) {
            transactionDir.mkdirs();
        }
    }

    public void save(Transaction tx) {
        File file = new File(transactionDir, tx.getTransactionId() + ".json");
        JsonSerializer.writeToFile(file, tx);
    }

    public Transaction findById(String transactionId) {
        File file = new File(transactionDir, transactionId + ".json");
        if (!file.exists())
            return null;
        return JsonSerializer.readFromFile(file, Transaction.class);
    }
}

```

```

package com.hospital.service;

import com.hospital.model.*;
import com.hospital.repository.impl.InventoryRepository;
import com.hospital.repository.impl.TransactionRepository;

import java.util.List;
import java.util.Optional;

public class InventoryService {

    private final InventoryRepository inventoryRepository;
    private final TransactionRepository transactionRepository;

    public InventoryService(InventoryRepository inventoryRepository,
        TransactionRepository transactionRepository) {
        this.inventoryRepository = inventoryRepository;
        this.transactionRepository = transactionRepository;
    }

    public List<MedicationStock> viewInventory() {
        return inventoryRepository.findAll();
    }

    public void withdrawMedication(String medicationId, int quantity) {
        Optional<MedicationStock> stockOpt =
inventoryRepository.findByMedicationId(medicationId);

        if (stockOpt.isEmpty()) {
            throw new IllegalArgumentException("Medication not found in inventory");
        }

        MedicationStock stock = stockOpt.get();

        if (stock.getQuantityAvailable() < quantity) {
            throw new IllegalStateException("Insufficient stock for withdrawal");
        }

        // Deduct stock
        stock.setQuantityAvailable(stock.getQuantityAvailable() - quantity);
        inventoryRepository.save(stock);

        // Log transaction
        Transaction tx = new Transaction("WITHDRAWAL", medicationId, quantity);

```

```

        transactionRepository.save(tx);
    }

    public void updateStock(String medicationId, int quantityAdded) {
        Optional<MedicationStock> stockOpt =
inventoryRepository.findByMedicationId(medicationId);
        if (stockOpt.isEmpty()) {
            throw new IllegalArgumentException("Medication not found in inventory.");
        }

        MedicationStock stock = stockOpt.get();
        stock.setQuantityAvailable(stock.getQuantityAvailable() + quantityAdded);
        inventoryRepository.save(stock);

        Transaction tx = new Transaction("RESTOCK", medicationId, quantityAdded);
        transactionRepository.save(tx);
    }

    public void addMedication(String medicationId, String name, int initialQuantity, int
reorderThreshold) {
        if (inventoryRepository.findByMedicationId(medicationId).isPresent()) {
            throw new IllegalStateException("Medication already exists in inventory");
        }

        MedicationStock stock = new MedicationStock(medicationId, name, initialQuantity,
reorderThreshold);
        inventoryRepository.save(stock);

        Transaction tx = new Transaction("ADD_MEDICATION", medicationId, initialQuantity);
        transactionRepository.save(tx);
    }

    public boolean isBelowThreshold(String medicationId) {
        Optional<MedicationStock> stockOpt =
inventoryRepository.findByMedicationId(medicationId);
        return stockOpt.map(stock -> stock.getQuantityAvailable() < stock.getReorderThreshold())
            .orElse(false);
    }
}

```